



# Passeio Turístico Máximo - Caminho Mais Longo

## IC0004 - Algoritmos e Grafos

|                   |                                                         |
|-------------------|---------------------------------------------------------|
| <b>Alunos</b>     | Yves Luan do Rosário Moura e Patrick César Santos Silva |
| <b>Disciplina</b> | IC0004 - Algoritmos e Grafos                            |
| <b>Professor</b>  | George Lima                                             |
| <b>Data</b>       | 01/07/2026                                              |

## Parte A - Complexidade e Redução Polinomial

O objetivo aqui é mostrar que o Caminho Mais Longo é NP-Difícil. Para isso trabalhamos com a versão de decisão do problema e construímos uma redução polinomial a partir do Caminho Hamiltoniano, que já se sabe ser NP-Completo. Usamos a definição de redução vista em aula. Basicamente, precisamos de uma função que pegue qualquer instância do Hamiltoniano e produza, em tempo polinomial, uma instância do Caminho Mais Longo, de um jeito que a resposta de uma seja SIM se, e somente se, a resposta da outra também for. É a relação que o enunciado escreve como  $\text{Caminho Hamiltoniano} \leq_p \text{Caminho Mais Longo}$ .

**Caminho Mais Longo (decisão).** Dados  $G=(V,E)$  não direcionado com pesos  $w(u,v) > 0$ , vértices  $S$  e  $D$  e um inteiro  $k$ , existe caminho simples de  $S$  a  $D$  com soma de pesos maior ou igual a  $k$ ?

**Caminho Hamiltoniano (decisão).** Dado  $G=(V,E)$  não direcionado, existe caminho simples que passa por todos os  $n=|V|$  vértices uma única vez?

## Construção da redução (transformação da entrada)

A primeira dificuldade apareceu antes mesmo da prova. O Caminho Hamiltoniano não diz onde o caminho começa nem onde termina, e o Caminho Mais Longo precisa de uma origem  $S$  e um destino  $D$  fixos. A saída que encontramos foi criar dois vértices artificiais para cumprir essa função, que no fundo são o hotel e a praça da história do enunciado.

Seja  $G=(V, E)$  uma instância qualquer do Caminho Hamiltoniano, com  $n=|V|$ . A partir dela montamos uma única instância  $(G', S, D, k)$  do Caminho Mais Longo assim:

- $G'$  começa como uma cópia de  $G$ , com os mesmos vértices e arestas;
- entram dois vértices novos,  $s^*$  e  $d^*$ ;
- $s^*$  é ligado a todos os vértices de  $V$ , e  $d^*$  também (sem aresta entre  $s^*$  e  $d^*$ );
- todas as arestas de  $G'$  recebem peso 1, tanto as antigas quanto as novas;
- e a instância final usa  $S = s^*$ ,  $D = d^*$  e  $k = n + 1$ .

Isso tudo é barato de construir. São só 2 vértices e  $2n$  arestas a mais, fora atribuir peso 1 às  $O(n^2)$  arestas, então a transformação claramente roda em tempo polinomial. Resolvida a primeira exigência da definição, falta mostrar que as respostas coincidem.

## Equivalência das respostas

**Ida.** Suponha que  $G$  tem um caminho hamiltoniano  $v_1, \dots, v_n$ . Basta encaixar  $s^*$  numa ponta e  $d^*$  na outra que aparece o caminho  $s^*, v_1, \dots, v_n, d^*$  em  $G'$ , e ele é simples. As duas arestas novas existem porque  $s^*$  e  $d^*$  são vizinhos de todos os vértices de  $V$ , e o trecho do meio já existia em  $G$ . São  $n+1$  arestas no total, cada uma pesando 1, o que dá soma exatamente  $n+1 = k$ . O Caminho Mais Longo responde "sim".

**Volta.** Agora suponha que existe um caminho simples de  $s^*$  a  $d^*$  em  $G'$  com soma maior ou igual a  $n+1$ . Toda aresta vale 1, então esse caminho usa pelo menos  $n+1$  arestas, isto é, pelo menos  $n+2$  vértices. Acontece que  $G'$  tem exatamente  $n+2$  vértices, e caminho simples não repete vértice nenhum. Conclusão, o caminho é obrigado a passar por todos, com  $s^*$  e  $d^*$  nas pontas e os  $n$  vértices de  $V$  no meio, em alguma ordem  $v_1, \dots, v_n$ . E os pares consecutivos desse miolo só podem estar ligados por arestas que já eram de  $G$ , porque as arestas que criamos encostam

sempre em `s*` ou em `d*`. Ou seja, `v1, ..., vn` é um caminho hamiltoniano em `G`, e o Hamiltoniano também responde "sim".

Com a transformação polinomial e a resposta preservada nos dois sentidos, o Caminho Hamiltoniano se reduz polinomialmente ao Caminho Mais Longo. E como o Hamiltoniano é NP-Completo, o Caminho Mais Longo é **NP-Difícil**. Dito de outro jeito, um algoritmo polinomial para ele resolveria o Hamiltoniano também, era só aplicar a transformação antes. Aproveitamos para registrar mais uma observação. Dado um caminho candidato como certificado, dá para conferir em tempo linear se ele é simples, se sai de `S`, chega em `D` e soma pelo menos `k`. Então a versão de decisão também está em NP e, juntando os dois fatos, ela é NP-Completa.

Uma pergunta que fica é por que o Caminho Mais Curto é fácil e o Mais Longo não. A diferença está na subestrutura ótima, aquela propriedade de que qualquer pedaço de um caminho mínimo também é mínimo, que é justamente o que o Dijkstra explora para montar a solução aos poucos. No mais longo simples ela quebra. Como não pode repetir vértice, cada escolha interfere nas seguintes, e uma decisão que parece boa agora pode estragar o resto da rota. É daí que vem a dificuldade.

---

## Parte B - Implementação e Experimentos

---

As duas abordagens foram implementadas em Python, sempre sobre grafos completos com pesos inteiros sorteados entre 1 e 100. Fixamos a semente do gerador (`random.seed(42)`) para qualquer um conseguir repetir o experimento e chegar nos mesmos números. A origem é sempre o vértice 0 e o destino é o vértice `n-1`. Os tempos abaixo saíram de um desktop com Ryzen 5 3500X e 16 GB de RAM, com Python 3.13 rodando no WSL2 (Ubuntu).

### Solução exata (backtracking)

A solução exata é um DFS recursivo que percorre todos os caminhos possíveis de `S` até `D` guardando o melhor. O ponto que exige cuidado é desmarcar o vértice quando a recursão volta, senão ele não pode ser reaproveitado em outras rotas e a busca deixa de ser completa. O melhor peso e o melhor caminho ficam em variáveis globais que vão sendo atualizadas ao longo da busca. Também impusemos um limite de 20 segundos, porque em grafo grande a busca simplesmente não acaba (nesse caso a função devolve `None`).

caminho\_mais\_longo.py

```
def buscar(g, n, atual, destino, visitado, peso, caminho, inicio, limite):  
    global melhor_peso, melhor_caminho, estourou  
    if estourou:  
        return  
    if time.time() - inicio > limite:  
        estourou = True  
        return  
    if atual == destino:  
        if peso > melhor_peso:  
            melhor_peso = peso  
            melhor_caminho = caminho[:]  
        return  
    for prox in range(n):  
        if not visitado[prox] and g[atual][prox] > 0:  
            visitado[prox] = True  
            caminho.append(prox)  
            buscar(g, n, prox, destino, visitado, peso + g[atual][prox],  
                  caminho, inicio, limite)  
            caminho.pop()  
            visitado[prox] = False # desmarca na volta  
  
def resolver_exato(g, n, origem, destino, limite=20.0):  
    global melhor_peso, melhor_caminho, estourou  
    melhor_peso = -1  
    melhor_caminho = None  
    estourou = False  
    visitado = [False] * n  
    visitado[origem] = True  
    buscar(g, n, origem, destino, visitado, 0, [origem], time.time(), limite)  
    if estourou:  
        return None, None  
    return melhor_peso, melhor_caminho
```

## Solução gulosa (heurística)

Já o guloso quase não tem o que explicar. Sai da origem e fica repetindo a mesma regra, escolher o vizinho ainda não visitado ligado pela aresta mais pesada. O processo para quando encontra o destino ou quando não sobra vizinho disponível, o famoso beco sem saída.

caminho\_mais\_longo.py

```
def resolver_guloso(g, n, origem, destino):
    visitado = [False] * n
    visitado[origem] = True
    atual = origem
    peso = 0
    caminho = [origem]

    while atual != destino:
        escolhido = -1
        maior = -1

        for prox in range(n):
            if not visitado[prox] and g[atual][prox] > maior:
                maior = g[atual][prox]
                escolhido = prox

        if escolhido == -1:
            return None, caminho # beco sem saída

        visitado[escolhido] = True
        peso = peso + maior
        caminho.append(escolhido)
        atual = escolhido

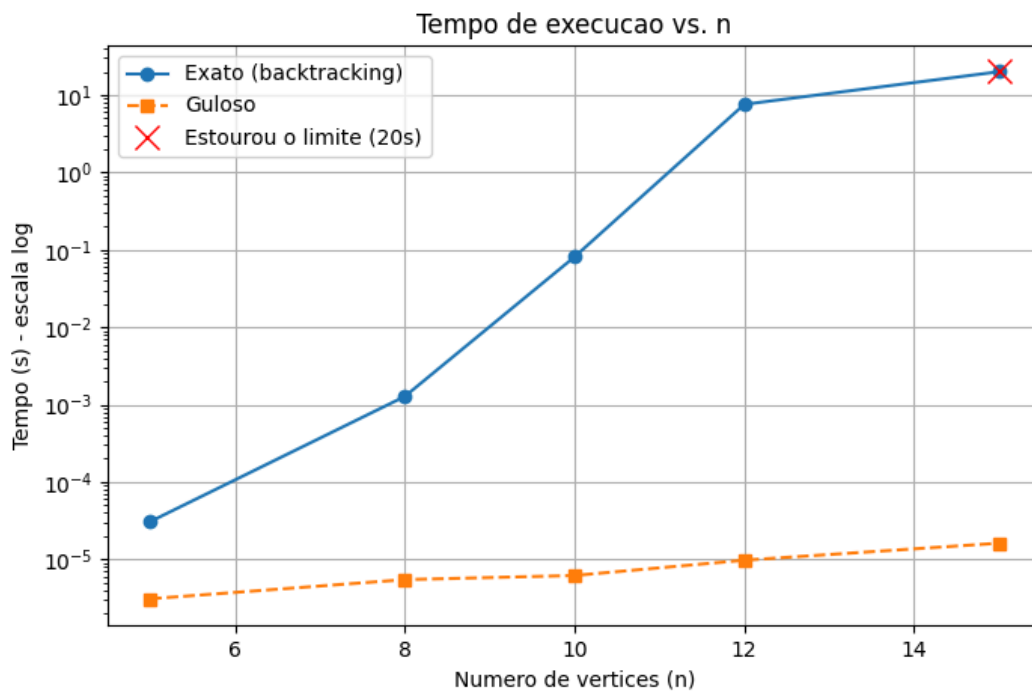
    return peso, caminho
```

O resto do código, com a geração dos grafos aleatórios, o laço dos experimentos e a montagem dos gráficos, está no arquivo [src/caminho\\_mais\\_longo.py](#), entregue junto com este relatório.

## Tempo de execução

| n | Exato   | Guloso   |
|---|---------|----------|
| 5 | 0,03 ms | 0,003 ms |
| 8 | 1,3 ms  | 0,005 ms |

| n  | Exato         | Guloso   |
|----|---------------|----------|
| 10 | 81,1 ms       | 0,006 ms |
| 12 | 7,58 s        | 0,010 ms |
| 15 | parou em 20 s | 0,016 ms |



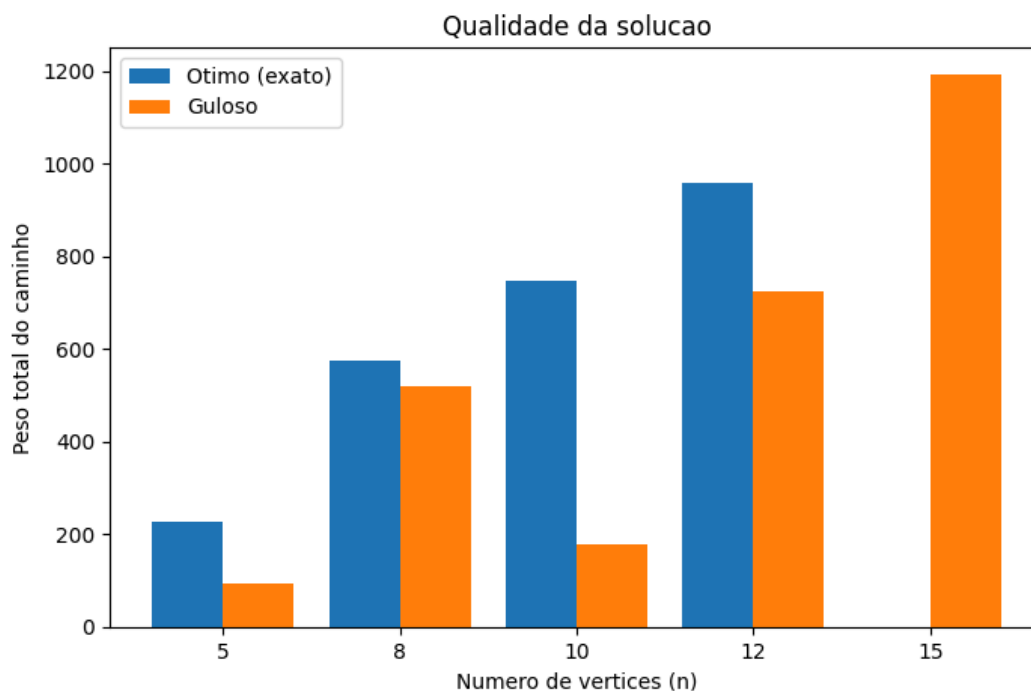
O gráfico usa escala logarítmica no eixo do tempo. Mesmo assim a curva do exato sobe quase como uma reta, sinal de crescimento explosivo, o que bate com a teoria, afinal num grafo completo o número de caminhos entre S e D é da ordem de  $(n-2)!$ . A linha do guloso, em comparação, mal sai do zero.

E onde a força bruta deixa de ser viável? No nosso computador,  $n=12$  já custou quase 8 segundos, e em  $n=15$  a execução passou do limite de 20 segundos e precisou ser interrompida. Em algum ponto entre 12 e 15 vértices, portanto, o método exato se torna impraticável. O guloso, para efeito de comparação, resolve o mesmo  $n=15$  em microssegundos.

## Qualidade da solução

| n | Ótimo | Guloso | Quanto pior |
|---|-------|--------|-------------|
| 5 | 227   | 95     | 58,1%       |

| n  | Ótimo        | Guloso | Quanto pior |
|----|--------------|--------|-------------|
| 8  | 574          | 521    | 9,2%        |
| 10 | 747          | 180    | 75,9%       |
| 12 | 960          | 726    | 24,4%       |
| 15 | não terminou | 1192   | -           |



Nos casos em que a força bruta terminou, a distância do guloso para o ótimo variou de 9% a 76%, sem padrão nenhum. Os piores cenários foram  $n=5$  e  $n=10$ . Neles a aresta mais pesada saindo da origem levava praticamente direto ao destino, o guloso acabou escolhendo justamente esse caminho e o passeio terminou curtíssimo, cobrindo só uma pequena parte do grafo.

O caso  $n=10$  ilustra bem esse comportamento. O backtracking achou  $0 \rightarrow 1 \rightarrow 7 \rightarrow 8 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 9$ , um caminho que visita os 10 vértices e soma 747. O guloso saiu do 0, foi para o vértice 7 por causa da aresta mais pesada e do 7 seguiu diretamente para o destino 9, ficando no  $0 \rightarrow 7 \rightarrow 9$ , de peso 180. Terminou cedo demais e deixou de explorar quase todo o grafo.

E por que o guloso erra tanto? O objetivo é maximizar a soma de um caminho sem repetir vértice. Escolher sempre a maior aresta do momento tem dois efeitos colaterais. Pode levar cedo demais ao destino, encerrando um caminho curto, e pode

gastar um vértice que faria muita falta numa rota maior mais adiante. Sem nunca voltar atrás, o algoritmo fica preso nesse ótimo local. Na prática, o problema não tem a propriedade de escolha gulosa que sustenta esse tipo de heurística em outros contextos, o que era de se esperar de um problema NP-Difícil.

## Resumindo o trade-off

- **Exato:** garante o melhor caminho sempre, só que o custo cresce na casa de  $(n-2)!$  e, em grafos completos, já fica inviável entre  $n=12$  e  $15$ .
- **Guloso:** roda em  $O(n^2)$  no pior caso e devolve alguma rota com custo quase nulo, mas sem nenhuma garantia de qualidade. Pode errar bastante e às vezes nem chega ao destino.

## Referências Bibliográficas

1. CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. Cambridge, MA: MIT Press, 2009.
2. GAREY, M. R.; JOHNSON, D. S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W. H. Freeman, 1979.
3. SIPSER, M. *Introduction to the Theory of Computation*. 3. ed. Boston: Cengage Learning, 2013.
4. KARP, R. M. Reducibility Among Combinatorial Problems. In: MILLER, R. E.; THATCHER, J. W. (Eds.). *Complexity of Computer Computations*. New York: Plenum Press, 1972. p. 85-103.
5. LIMA, G. *Notas e slides de aula: Programação Dinâmica, Algoritmos Gulosos, Backtracking e Branch and Bound, e Introdução às classes P, NP e NP-Completo*. Instituto de Computação, UFBA, 2025.
6. HUNTER, J. D. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, v. 9, n. 3, p. 90-95, 2007.